

Defensa Técnica — Yandri Alcivar

Proyecto: Alpha Inmobiliaria
Rol: Backend Lead / DevOps
Sprint: 1

1. Equipo y Repartición

Yandri Alcivar — Backend Lead / DevOps

Área	Tecnologías Entregadas
Monorepo	Node.js, Express 5, TypeScript 6, npm workspaces
Base de Datos	Prisma 7, PostgreSQL (Supabase), @prisma/adapters , pg
CRUDs	22 endpoints REST con arquitectura hexagonal
Validación	Zod 4 (schemas de validación para todas las entradas)
Seguridad	bcryptjs (hash de contraseñas), JWT (jsonwebtoken)
Archivos	Cloudinary SDK, Multer (subida de imágenes)
Git	Feature branches, Pull Requests, merges a main
Documentación	14 archivos markdown en docs/

Historias de usuario que soporta: HU-002 (login), HU-019 (JWT), HU-026 (perfil asesor), HU-033 (deploy)

Tareas Técnicas (TTs) ejecutadas: 9 (TT-001 a TT-008 + TT-005.5)

Jhonny — Frontend Público

Tecnologías: React 19, Vite, TypeScript, Tailwind CSS, React Router, Axios
Historias: HU-003 (catálogo), HU-004 (búsqueda/filtros), HU-005 (detalle), HU-010 (contacto), HU-013 (notificaciones), HU-015 (destacadas), HU-020 (favoritos), HU-021 (compartir), HU-023 (citas), HU-028 (push), HU-034 (SEO)

Kelvin — Admin Panel

Tecnologías: React 19, Vite, TypeScript, React Table, React Hook Form, Cloudinary Widget
Historias: HU-006 (CRUD propiedades admin), HU-007 (galería), HU-012 (panel admin), HU-017 (imágenes dispositivo), HU-022 (reportes), HU-029 (exportar), HU-031 (tours virtuales)

Allan — Mapas / Frontend

Tecnologías: React 19, Vite, TypeScript, Leaflet, Mapbox
Historias: HU-001 (registro), HU-008 (mapa interactivo), HU-011 (perfil asesor), HU-014 (reset password), HU-

016 (testimonios), HU-018 (responsive), HU-024 (valoraciones), HU-025 (términos), HU-030 (Google Maps), HU-035 (modo oscuro)

Ricardo — Chatbot IA

Tecnologías: React 19, Vite, TypeScript, GROQ API (gratuito)
Historias: HU-009 (chatbot IA), HU-027 (chat en vivo), HU-032 (sugerencias IA avanzadas)

2. Stack Tecnológico del Backend

Capa	Tecnología	Versión
Runtime	Node.js	22+
Lenguaje	TypeScript	6.x
Framework HTTP	Express	5.x
ORM	Prisma	7.x
Driver DB	@prisma/adapters + pg	8.x
Base de Datos	PostgreSQL (Supabase)	15+
Validación	Zod	4.x
Autenticación	JWT (jsonwebtoken)	9.x
Contraseñas	bcryptjs	3.x
Imágenes	Cloudinary	2.x
Archivos	Multer	2.x

3. Dependencias del Backend (mi parte)

Producción — 12 paquetes

Paquete	Versión	Categoría	Función en mi código
express	^5.2.1	Framework	Creación de rutas, middlewares, controllers. Usado en server.ts y todas las rutas
@prisma/client	^7.8.0	ORM	Consultas tipadas a PostgreSQL. Usado en todos los repositorios
@prisma/adapters + pg	^7.8.0	ORM	Adaptador requerido por Prisma 7 para conectar con PostgreSQL
prisma	^7.8.0	ORM	CLI para migraciones: prisma migrate dev, prisma generate
pg	^8.22.0	Driver	Conexión nativa a PostgreSQL usada internamente por el adapter

Paquete	Versión	Categoría	Función en mi código
<code>zod</code>	<code>^4.4.3</code>	Validación	Esquemas de validación en <code>propiedad.validation.ts</code> , <code>asesor.validation.ts</code> , <code>auth.validation.ts</code> , <code>mensaje.validation.ts</code>
<code>bcryptjs</code>	<code>^3.0.3</code>	Seguridad	Hash de contraseñas en <code>AsesorService.ts</code> y comparación en <code>AuthService.ts</code>
<code>jsonwebtoken</code>	<code>^9.0.3</code>	Autenticación	Generación de tokens en <code>AuthService.ts</code> y verificación en <code>authMiddleware.ts</code>
<code>cloudinary</code>	<code>^2.10.0</code>	Archivos	Subida de imágenes en <code>ImagenService.ts</code> mediante <code>upload_stream</code>
<code>multer</code>	<code>^2.2.0</code>	Archivos	Procesamiento de multipart/form-data en <code>upload.ts</code> middleware
<code>cors</code>	<code>^2.8.6</code>	HTTP	Habilitar peticiones cross-origin en <code>server.ts</code>
<code>dotenv</code>	<code>^17.4.2</code>	Config	Carga de variables de entorno desde <code>.env</code> en <code>server.ts</code> y <code>prisma.ts</code>

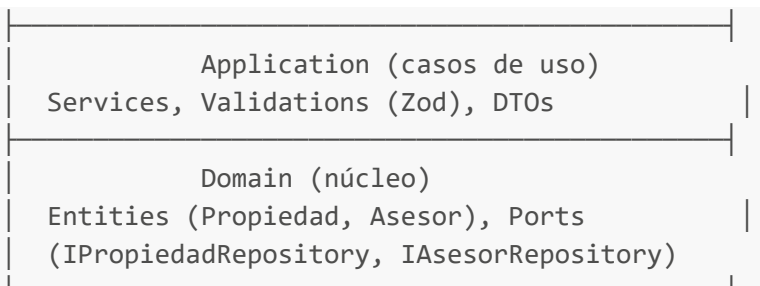
Desarrollo — 9 paquetes

Paquete	Versión	Función
<code>typescript</code>	<code>^6.0.3</code>	Compilador TS → JS con configuración en <code>tsconfig.json</code>
<code>tsx</code>	<code>^4.22.4</code>	Ejecución directa de TypeScript con <code>npm run dev</code> (watch mode)
<code>@types/node</code>	<code>^26.0.1</code>	Tipos para APIs nativas de Node.js
<code>@types/express</code>	<code>^5.0.6</code>	Tipos para Request, Response, NextFunction
<code>@types/cors</code>	<code>^2.8.19</code>	Tipos para el middleware CORS
<code>@types/bcryptjs</code>	<code>^2.4.6</code>	Tipos para bcrypt (hash, compare)
<code>@types/jsonwebtoken</code>	<code>^9.0.10</code>	Tipos para jwt.sign, jwt.verify
<code>@types/multer</code>	<code>^2.1.0</code>	Tipos para Multer.File, memoryStorage
<code>@types/pg</code>	<code>^8.20.0</code>	Tipos para el driver PostgreSQL

4. Arquitectura Hexagonal (Puertos y Adaptadores)

Es la base de todo mi backend. Separa el código en 3 capas:

```
Infrastructure (adaptadores)
Controllers, Repositories, Prisma, Multer,
Cloudinary, Middlewares, Routes
```



Principio de Inversión de Dependencias (aplicado en TT-005.5)

✗ ANTES (violación):

Service.ts → import repositorio concreto (singleton)

✓ DESPUÉS (corregido):

```
Routes.ts (wiring) → new Repositorio()
                    → new Servicio(repositorio)    ← solo conoce interfaz
                    → new Controlador(servicio)    ← solo conoce servicio
```

Inyección de Dependencias en cada ruta

```
// asesor.routes.ts – ejemplo del wiring hexagonal real
const repository = new AsesorRepository()
const service = new AsesorService(repository) // no importa repositorio
const controller = new AsesorController(service) // no importa servicio

router.get('/asesores', controller.listar)
```

5. Tareas Técnicas Completadas

TT-001: Configuración del Monorepo

Archivos creados: `package.json` (raíz), `apps/backend/package.json`, `tsconfig.json`, `.env`, `src/server.ts`, `.gitignore`

Logros: Monorepo npm workspaces, Express 5 + TypeScript + tsx, health check `/api/health`, repositorio GitHub, workflow feature branch → PR → main

TT-002: Esquema Prisma y Modelo de Datos

Archivos: `prisma/schema.prisma`, `prisma/migrations/20260628042727_init/migration.sql`

Entidades (7): User, Asesor, Propiedad, Ubicacion, Imagen, Mensaje

Enums (5): Rol, TipoPropiedad, TipoTransaccion, EstadoPropiedad, TipoMensaje

Decisión SOLID: User y Asesor separados — User solo auth, Asesor datos profesionales

TT-003: Conexión PostgreSQL (Supabase)

Archivos: `src/infrastructure/database/prisma.ts`

Tecnologías: `@prisma/adapters-pg + pg` + Transaction Pooler de Supabase + IPv4 add-on

Logros: Conexión exitosa, health check verifica DB, migración inicial ejecutada

TT-004: CRUD de Propiedades (7 endpoints)

Endpoints:

Método	Ruta
GET	<code>/api/propiedades</code>
GET	<code>/api/propiedades/destacadas</code>
GET	<code>/api/propiedades/:id</code>
POST	<code>/api/propiedades</code>
PUT	<code>/api/propiedades/:id</code>
PATCH	<code>/api/propiedades/:id/estado</code>
DELETE	<code>/api/propiedades/:id</code>

Archivos (11): 1 puerto, 1 validación Zod, 1 service, 3 DTOs request, 1 DTO response, 1 repository, 1 controller, 1 routes, 1 middleware

Problema resuelto: `req.params.id` tipado como `string | string[]` — creado helper `getId(req)`

TT-005: CRUD de Asesores (5 endpoints)

Endpoints: GET listar (+ `_count` propiedades/mensajes), GET por ID, POST crear, PUT actualizar, DELETE eliminar

Archivos (9): Puerto, validación, service, 2 DTOs request, 1 DTO response, repository, controller, routes

Transacción: `prisma.$transaction` crea User + Asesor atómicamente

Cascada DELETE: mensajes → propiedades → Asesor → User (por FK)

TT-005.5: Refactor Arquitectura Hexagonal

5 violaciones corregidas:

#	Problema	Solución
a	Service importaba repositorio concreto	Constructor recibe interfaz
b	Puerto importaba Prisma	Eliminado import fantasma
c	Repository sin <code>implements</code>	Agregado <code>implements I*Repository</code>
d	Todo <code>any</code> sin entidades	Creadas <code>domain/entities/</code>
e	Singletons no testables	DI por constructor

TT-006: API de Mensajes + Cloudinary (8 endpoints)

Mensajes (5): POST crear, GET por asesor, PATCH leer, PATCH archivar, DELETE

Imágenes (3): POST subir 1, POST subir múltiples, DELETE eliminar de Cloudinary + DB

Archivos (13): incluye `cloudinary/config.ts`, `middlewares/upload.ts` (Multer memoryStorage, 5MB, JPEG/PNG/WebP)

Fix adicional: `PropiedadRepository.create` corregido con `connect` para Prisma 7

TT-007: Autenticación JWT (2 endpoints)

Endpoints: POST `/api/auth/login` (público), GET `/api/auth/me` (protegido)

Flujo: `Zod` → `findByEmail` → `bcrypt.compare` → `jwt.sign` → { token, user }

Middleware: `requireAuth` (protege rutas), `requireRol('admin')` (restringe por rol)

TT-008: Búsqueda y Filtros Avanzados

Filtros agregados: `banios`, `parqueos`, `sector`, `ciudad`, `metrajeMin`, `metrajeMax`, `ordenarPor`, `ordenDireccion`

Archivos modificados (3): `propiedad.validation.ts` (+8 campos), `PropiedadService.ts` (Zod en listar), `PropiedadRepository.ts` (filtros + `orderBy` dinámico)

6. Resumen de Endpoints Implementados

22 endpoints REST funcionales:

#	Método	Ruta	TT	Propósito
1	GET	<code>/api/health</code>	001	Health check con verificación DB
2	GET	<code>/api/propiedades</code>	004	Listar propiedades (con filtros)
3	GET	<code>/api/propiedades/destacadas</code>	004	Propiedades destacadas
4	GET	<code>/api/propiedades/:id</code>	004	Detalle de propiedad
5	POST	<code>/api/propiedades</code>	004	Crear propiedad
6	PUT	<code>/api/propiedades/:id</code>	004	Actualizar propiedad
7	PATCH	<code>/api/propiedades/:id/estado</code>	004	Cambiar estado
8	DELETE	<code>/api/propiedades/:id</code>	004	Eliminar propiedad
9	GET	<code>/api/asesores</code>	005	Listar asesores
10	GET	<code>/api/asesores/:id</code>	005	Detalle asesor
11	POST	<code>/api/asesores</code>	005	Crear asesor
12	PUT	<code>/api/asesores/:id</code>	005	Actualizar asesor
13	DELETE	<code>/api/asesores/:id</code>	005	Eliminar asesor
14	POST	<code>/api/mensajes</code>	006	Enviar mensaje
15	GET	<code>/api/mensajes/asesor/:asesorId</code>	006	Listar mensajes
16	PATCH	<code>/api/mensajes/:id/leer</code>	006	Marcar leído

#	Método	Ruta	TT	Propósito
17	PATCH	/api/mensajes/:id/archivar	006	Archivar mensaje
18	DELETE	/api/mensajes/:id	006	Eliminar mensaje
19	POST	/api/propiedades/:id/imagenes	006	Subir imagen
20	DELETE	/api/imagenes/:id	006	Eliminar imagen
21	POST	/api/auth/login	007	Iniciar sesión
22	GET	/api/auth/me	007	Obtener usuario actual

7. Estructura de Archivos (mi código)

```

apps/backend/src/
├── server.ts                                ← Punto de entrada, monta todos los routers
├── domain/
│   ├── entities/                          ← 6 entidades de dominio
│   │   ├── Propiedad.ts
│   │   ├── Asesor.ts
│   │   ├── User.ts
│   │   ├── Ubicacion.ts
│   │   ├── Imagen.ts
│   │   └── Mensaje.ts
│   └── ports/                             ← Interfaces (puertos)
│       ├── IPropiedadRepository.ts
│       ├── IAsesorRepository.ts
│       ├── IMensajeRepository.ts
│       ├── IImagenRepository.ts
│       └── IUserRepository.ts
├── application/
│   ├── services/                          ← Casos de uso
│   │   ├── PropiedadService.ts
│   │   ├── AsesorService.ts
│   │   ├── MensajeService.ts
│   │   ├── ImagenService.ts
│   │   └── AuthService.ts
│   ├── validations/                      ← Schemas Zod
│   │   ├── propiedad.validation.ts
│   │   ├── asesor.validation.ts
│   │   ├── mensaje.validation.ts
│   │   └── auth.validation.ts
│   └── dtos/                             ← Data Transfer Objects
│       ├── requests/
│       └── responses/
├── infrastructure/
│   ├── database/prisma.ts                ← Cliente Prisma + adapter-pg
│   └── cloudinary/config.ts              ← Configuración Cloudinary

```

```

├── middlewares/
│   ├── validateId.ts      ← Helper getId(req)
│   ├── upload.ts         ← Multer memoryStorage
│   └── authMiddleware.ts  ← JWT requireAuth + requireRol
├── repositories/
│   ├── PropiedadRepository.ts
│   ├── AsesorRepository.ts
│   ├── MensajeRepository.ts
│   ├── ImagenRepository.ts
│   └── UserRepository.ts
│   ← Implementaciones Prisma
├── controllers/          ← Handlers HTTP
│   ├── PropiedadController.ts
│   ├── AsesorController.ts
│   ├── MensajeController.ts
│   ├── ImagenController.ts
│   └── AuthController.ts
├── routes/               ← Rutas + wiring DI
│   ├── propiedad.routes.ts
│   ├── asesor.routes.ts
│   ├── mensaje.routes.ts
│   ├── imagen.routes.ts
│   └── auth.routes.ts
└── prisma/
    └── schema.prisma      ← 7 entidades, 5 enums

```

Total de archivos creados por Yandri: ~50 archivos fuente

8. Decisiones Técnicas Clave (para defender)

Decisión	Opción Elegida	Por qué
Monorepo	npm workspaces	Un solo <code>npm install</code> , dependencias compartidas, organización clara
User + Asesor separados	Modelos independientes	Principio SOLID: User solo auth, Asesor datos profesionales
Prisma 7	<code>@prisma/adapters-pg</code>	Requerido por Prisma 7, más eficiente que URL connection string
Supabase Pooler	Transaction Pooler + IPv4	Maneja múltiples conexiones, evita error de IP
Hexagonal Architecture	Puertos y Adaptadores	Aislamiento de capas, testabilidad, cambio de ORM sin tocar negocio
Validación Zod	Schemas en application layer	Datos inválidos nunca llegan a DB
bcryptjs	10 salt rounds	Estándar de la industria para hash de contraseñas
JWT	7 días de expiración	Balance entre seguridad y experiencia de usuario

Decisión	Opción Elegida	Por qué
Cloudinary	Upload stream desde buffer	No guarda archivos en disco, eficiente en servidores serverless
Multer memoryStorage	Buffer en RAM	Ideal para Cloudinary, evita escritura a disco
Git workflow	feature → PR → main	Código revisado antes de merge, rama main siempre estable

Documento generado el 29/06/2026 — Sprint 1 completado